



Panduan Lengkap: Membuat Boilerplate Generator Python dengan Library yang Direkomendasikan

Baik! Sekarang saya akan memandu Anda step-by-step membuat tool boilerplate generator `zul` menggunakan **Typer**, **Copier**, **InquirerPy**, **Rich**, **Jinja2**, dan **Pydantic Settings**. Kita akan membuat implementasi yang production-ready dan mudah di-extend.

Langkah 1: Setup Project

1.1 Buat Struktur Directory

```
mkdir zul-cli
cd zul-cli

# Buat struktur project
mkdir -p zul/{commands,templates/{agent,api,cli},utils}
touch zul/__init__.py
touch zul/cli.py
touch zul/commands/{__init__.py,build.py,init.py}
touch zul/utils/{__init__.py,file_generator.py,config.py}
```

1.2 Buat pyproject.toml

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "zul"
version = "0.1.0"
description = "Modern CLI tool for generating Python project boilerplates"
authors = [{name = "Your Name", email = "[email protected]"}]
requires-python = ">=3.9"
dependencies = [
    "typer[all]>=0.12.0",      # CLI framework (includes Rich)
    "inquirerpy>=0.3.4",     # Interactive prompts
    "jinja2>=3.1.0",         # Template rendering
    "pydantic>=2.0.0",       # Data validation
    "pydantic-settings>=2.0.0", # Settings management
]

[project.optional-dependencies]
dev = [
```

```

    "pytest>=7.0.0",
    "black>=23.0.0",
    "ruff>=0.1.0",
]

[project.scripts]
zul = "zul.cli:app"

[tool.hatch.build.targets.wheel]
packages = ["zul"]

```

1.3 Install Dependencies

```

# Buat virtual environment
python -m venv venv
source venv/bin/activate # Di Windows: venv\Scripts\activate

# Install project dalam editable mode
pip install -e .

```

Langkah 2: Buat Main CLI dengan Typer

File: zul/cli.py

```

"""
Zul CLI - Modern Python Boilerplate Generator
"""

import typer
from rich.console import Console
from rich import print as rprint
from typing_extensions import Annotated

# Initialize Typer app
app = typer.Typer(
    name="zul",
    help="🐍 Modern Python boilerplate generator",
    add_completion=True,
    rich_markup_mode="rich"
)

console = Console()

@app.callback()
def callback():
    """
    Zul - Generate Python project boilerplates with ease! 🐍
    """
    pass

# Import commands (akan dibuat nanti)
from zul.commands import build, init

```

```
# Register subcommands
app.add typer(build.app, name="build")
app.add typer(init.app, name="init")

@app.command()
def version():
    """Show Zul version"""
    rprint("[bold cyan]Zul[/bold cyan] version [green]0.1.0[/green] ")

if __name__ == "__main__":
    app()
```

Langkah 3: Buat Build Command dengan InquirerPy

File: zul/commands/build.py

```
"""
Build command - Generate project from template
"""

import typer
from pathlib import Path
from rich.console import Console
from rich.panel import Panel
from rich.progress import Progress, SpinnerColumn, TextColumn
from InquirerPy import inquirer
from InquirerPy.validator import PathValidator
import time

from zul.utils.file_generator import FileGenerator

app = typer.Typer(help="Build a new project from template")
console = Console()

@app.command()
def agent(
    name: str = typer.Option(None, "--name", "-n", help="Project name"),
    output: Path = typer.Option(".", "--output", "-o", help="Output directory"),
    interactive: bool = typer.Option(True, "--interactive/--no-interactive", help="Interactive"),
):
    """
    Build an AI Agent project

    Creates a complete AI agent application with:
    - Agent base classes
    - Configuration management
    - Tool integration
    - Testing setup
    """

    console.print(Panel.fit(
        "[bold cyan]Building AI Agent Project[/bold cyan] ",

```

```

        border_style="cyan"
    ))

# Interactive prompts jika tidak ada name
if interactive and not name:
    name = inquirer.text(
        message="What's your project name?",
        default="my_agent",
        validate=lambda x: x.isidentifier() or "Must be valid Python identifier"
    ).execute()

# Additional prompts
use_openai = inquirer.confirm(
    message="Include OpenAI integration?",
    default=True
).execute()

use_anthropic = inquirer.confirm(
    message="Include Anthropic integration?",
    default=False
).execute()

use_docker = inquirer.confirm(
    message="Include Docker configuration?",
    default=True
).execute()

use_tests = inquirer.confirm(
    message="Setup testing framework (pytest)?",
    default=True
).execute()
else:
    use_openai = True
    use_anthropic = False
    use_docker = True
    use_tests = True

# Validate project name
if not name.isidentifier():
    console.print(f"[red]Error:[/red] '{name}' is not a valid Python package name")
    raise typer.Exit(1)

# Create project directory
project_path = output / name
if project_path.exists():
    overwrite = inquirer.confirm(
        message=f"Directory {project_path} already exists. Overwrite?",
        default=False
    ).execute()

    if not overwrite:
        console.print("[yellow]Aborted[/yellow]")
        raise typer.Exit(0)

# Generate project with progress
context = {

```

```

        'project_name': name,
        'project_name_title': name.replace('_', ' ').title(),
        'package_name': name.lower().replace('-', '_'),
        'use_openai': use_openai,
        'use_anthropic': use_anthropic,
        'use_docker': use_docker,
        'use_tests': use_tests,
        'version': '0.1.0',
    }

    with Progress(
        SpinnerColumn(),
        TextColumn("[progress.description]{task.description}"),
        console=console
    ) as progress:
        task = progress.add_task("Generating project files...", total=None)

        generator = FileGenerator('agent')
        generator.generate(project_path, context)

        progress.update(task, completed=True)

    # Success message
    console.print(f"\n[green]✓[/green] Project '{name}' created successfully! 🎉\n")

    # Next steps
    console.print(Panel(
        f"""[bold]Next steps:[/bold]

1. [cyan]cd {name}[/cyan]
2. [cyan]python -m venv venv[/cyan]
3. [cyan]source venv/bin/activate[/cyan] # Windows: venv\Scripts\activate
4. [cyan]pip install -e .[/cyan]
5. [cyan]cp .env.example .env[/cyan]
6. Edit [yellow].env[/yellow] and add your API keys
7. [cyan]python -m {context['package_name']}.main[/cyan]

[dim]Documentation: README.md[/dim]""",
        title="📖 Get Started",
        border_style="green"
    ))

@app.command()
def api(
    name: str = typer.Option(None, "--name", "-n", help="Project name"),
    output: Path = typer.Option(".", "--output", "-o", help="Output directory"),
    interactive: bool = typer.Option(True, "--interactive/--no-interactive", help="Interactive mode"),
):
    """
    🛠 Build a FastAPI project

    Creates a complete FastAPI application with:
    - API routes structure
    - Database integration
    - Authentication

```

```

- Testing setup
"""

console.print(Panel.fit(
    "[bold cyan]Building FastAPI Project[/bold cyan] ",
    border_style="cyan"
))

if interactive and not name:
    name = inquirer.text(
        message="What's your project name?",
        default="my_api",
        validate=lambda x: x.isidentifier() or "Must be valid Python identifier"
    ).execute()

    framework = inquirer.select(
        message="Choose your framework:",
        choices=[
            {"name": "FastAPI (Recommended)", "value": "fastapi"},
            {"name": "Flask", "value": "flask"},
        ],
        default="fastapi"
    ).execute()

    database = inquirer.select(
        message="Choose database:",
        choices=[
            {"name": "PostgreSQL + SQLAlchemy", "value": "postgresql"},
            {"name": "SQLite + SQLAlchemy", "value": "sqlite"},
            {"name": "MongoDB + Motor", "value": "mongodb"},
            {"name": "No database", "value": "none"},
        ],
        default="postgresql"
    ).execute()

    use_auth = inquirer.confirm(
        message="Include authentication (JWT)?",
        default=True
    ).execute()

    use_docker = inquirer.confirm(
        message="Include Docker configuration?",
        default=True
    ).execute()
else:
    framework = "fastapi"
    database = "postgresql"
    use_auth = True
    use_docker = True

context = {
    'project_name': name,
    'project_name_title': name.replace('_', ' ').title(),
    'package_name': name.lower().replace('-', '_'),
    'framework': framework,
    'database': database,

```

```

        'use_auth': use_auth,
        'use_docker': use_docker,
        'version': '0.1.0',
    }

    project_path = output / name

    with Progress(
        SpinnerColumn(),
        TextColumn("[progress.description]{task.description}"),
        console=console
    ) as progress:
        task = progress.add_task("Generating API project...", total=None)

        generator = FileGenerator('api')
        generator.generate(project_path, context)

        progress.update(task, completed=True)

    console.print(f"\n[green]✓[/green] API project '{name}' created successfully! 🎉\n")

@app.command()
def cli(
    name: str = typer.Option(None, "--name", "-n", help="Project name"),
    output: Path = typer.Option(".", "--output", "-o", help="Output directory")
):
    """
    🛠 Build a CLI tool project

    Creates a complete CLI application with:
    - Typer framework
    - Rich output
    - Subcommands structure
    - Testing setup
    """

    console.print("[cyan]Building CLI tool project...[/cyan]")

    if not name:
        name = inquirer.text(
            message="What's your CLI tool name?",
            default="my_cli"
        ).execute()

    context = {
        'project_name': name,
        'package_name': name.lower().replace('-', '_'),
        'version': '0.1.0',
    }

    project_path = output / name

    generator = FileGenerator('cli')
    generator.generate(project_path, context)

```

```
console.print(f"[green]✓[/green] CLI project '{name}' created!")
```

Langkah 4: Buat File Generator dengan Jinja2

File: zul/utils/file_generator.py

```
"""
File generation utilities using Jinja2
"""

from pathlib import Path
from jinja2 import Environment, PackageLoader, select_autoescape, FileSystemLoader
from rich.console import Console
import os

console = Console()

class FileGenerator:
    """Generate project files from Jinja2 templates"""

    def __init__(self, template_type: str):
        """
        Initialize generator with template type

        Args:
            template_type: Type of template (agent, api, cli)
        """
        self.template_type = template_type

        # Setup Jinja2 environment
        template_dir = Path(__file__).parent.parent / "templates" / template_type

        self.env = Environment(
            loader=FileSystemLoader(str(template_dir)),
            autoescape=select_autoescape(),
            trim_blocks=True,
            lstrip_blocks=True,
            keep_trailing_newline=True,
        )

        # Add custom filters
        self.env.filters['slugify'] = lambda x: x.lower().replace(' ', '_').replace('-', '')

    def generate(self, output_path: Path, context: dict):
        """
        Generate project files

        Args:
            output_path: Where to generate files
            context: Template context variables
        """
        output_path.mkdir(parents=True, exist_ok=True)

        # Get template structure definition
```

```

structure = self._get_structure(context)

# Create directories
for dir_path in structure['directories']:
    full_path = output_path / self._render_path(dir_path, context)
    full_path.mkdir(parents=True, exist_ok=True)

    # Create __init__.py for Python packages
    if not dir_path.startswith('.') and dir_path != 'tests':
        (full_path / '__init__.py').touch()

# Generate files from templates
for template_file, output_file in structure['files'].items():
    self._generate_file(template_file, output_path, output_file, context)

console.print(f"[green]Generated {len(structure['files'])} files[/green]")

def _get_structure(self, context: dict) -> dict:
    """Get project structure based on template type and context"""

    if self.template_type == 'agent':
        return {
            'directories': [
                context['package_name'],
                f"{context['package_name']}/agents",
                f"{context['package_name']}/tools",
                f"{context['package_name']}/utils",
                'tests',
                'configs',
                'data',
            ],
            'files': {
                'main.py.j2': f"{context['package_name']}/main.py",
                'config.py.j2': f"{context['package_name']}/config.py",
                'agents/base_agent.py.j2': f"{context['package_name']}/agents/base_agent.py",
                'tools/base_tool.py.j2': f"{context['package_name']}/tools/base_tool.py",
                'pyproject.toml.j2': 'pyproject.toml',
                'README.md.j2': 'README.md',
                '.env.example.j2': '.env.example',
                '.gitignore.j2': '.gitignore',
                'Dockerfile.j2': 'Dockerfile' if context.get('use_docker') else None,
                'docker-compose.yml.j2': 'docker-compose.yml' if context.get('use_docker') else None,
            }
        }

    elif self.template_type == 'api':
        return {
            'directories': [
                context['package_name'],
                f"{context['package_name']}/api",
                f"{context['package_name']}/api/routes",
                f"{context['package_name']}/models",
                f"{context['package_name']}/services",
                'tests',
            ],
            'files': {

```

```

        'main.py.j2': f"{context['package_name']}/main.py",
        'config.py.j2': f"{context['package_name']}/config.py",
        'pyproject.toml.j2': 'pyproject.toml',
        'README.md.j2': 'README.md',
    }
}

elif self.template_type == 'cli':
    return {
        'directories': [
            context['package_name'],
            f"{context['package_name']}/commands",
            'tests',
        ],
        'files': {
            'cli.py.j2': f"{context['package_name']}/cli.py",
            'pyproject.toml.j2': 'pyproject.toml',
            'README.md.j2': 'README.md',
        }
    }

return {'directories': [], 'files': {}}

def _render_path(self, path: str, context: dict) -> str:
    """Render path with Jinja2"""
    template = self.env.from_string(path)
    return template.render(**context)

def _generate_file(self, template_name: str, output_path: Path, output_file: str, cor
    """Generate a single file from template"""

    if output_file is None:
        return

    try:
        # Load and render template
        template = self.env.get_template(template_name)
        content = template.render(**context)

        # Write to file
        file_path = output_path / output_file
        file_path.parent.mkdir(parents=True, exist_ok=True)
        file_path.write_text(content, encoding='utf-8')

        console.print(f"  [dim]Created[/dim] {output_file}")

    except Exception as e:
        console.print(f"[red]Error generating {output_file}:[/red] {e}")

```

Langkah 5: Buat Template Files

File: zul/templates/agent/main.py.j2

```
"""
{{ project_name_title }}
Main entry point for the agent application.
"""

import logging
from {{ package_name }}.config import settings
from {{ package_name }}.agents.base_agent import Agent

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)

logger = logging.getLogger(__name__)

def main():
    """Main function to run the agent"""
    logger.info("Starting {{ project_name_title }}...")

    # Initialize agent
    agent = Agent(
        name="{{ project_name }}",
        model=settings.MODEL_NAME,
        temperature=settings.TEMPERATURE
    )

    # Example usage
    try:
        prompt = "Hello! Please introduce yourself."
        logger.info(f"Sending prompt: {prompt}")

        response = agent.run(prompt)
        print(f"\n Agent response:\n{response}\n")

    except Exception as e:
        logger.error(f"Error running agent: {e}")
        raise

if __name__ == "__main__":
    main()
```

File: zul/templates/agent/config.py.j2

```
"""
Configuration management for {{ project_name_title }}
"""
```

```

from pydantic_settings import BaseSettings, SettingsConfigDict
from typing import Optional

class Settings(BaseSettings):
    """Application settings with Pydantic validation"""

    # Model configuration
    MODEL_NAME: str = "gpt-4"
    TEMPERATURE: float = 0.7
    MAX_TOKENS: int = 1000

    {% if use_openai -%}
    # OpenAI API
    OPENAI_API_KEY: Optional[str] = None
    OPENAI_ORG_ID: Optional[str] = None
    {% endif -%}

    {% if use_anthropic -%}
    # Anthropic API
    ANTHROPIC_API_KEY: Optional[str] = None
    {% endif -%}

    # Application settings
    LOG_LEVEL: str = "INFO"
    DEBUG: bool = False
    ENVIRONMENT: str = "development"

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=True,
        extra="ignore"
    )

# Global settings instance
settings = Settings()

```

File: zul/templates/agent/agents/base_agent.py.j2

```

"""
Base Agent implementation for {{ project_name_title }}
"""

from typing import Optional, List, Dict, Any
import logging
{% if use_openai -%}
from openai import OpenAI
{% endif -%}
{% if use_anthropic -%}
from anthropic import Anthropic
{% endif -%}

from {{ package_name }}.config import settings

```

```

logger = logging.getLogger(__name__)

class Agent:
    """Base agent class for LLM interactions"""

    def __init__(
        self,
        name: str,
        model: str = "gpt-4",
        temperature: float = 0.7,
        max_tokens: int = 1000
    ):
        self.name = name
        self.model = model
        self.temperature = temperature
        self.max_tokens = max_tokens
        self.conversation_history: List[Dict[str, str]] = []

        {% if use_openai -%}
        # Initialize OpenAI client
        if settings.OPENAI_API_KEY:
            self.openai_client = OpenAI(api_key=settings.OPENAI_API_KEY)
        else:
            logger.warning("OpenAI API key not found")
            self.openai_client = None
        {% endif -%}

        {% if use_anthropic -%}
        # Initialize Anthropic client
        if settings.ANTHROPIC_API_KEY:
            self.anthropic_client = Anthropic(api_key=settings.ANTHROPIC_API_KEY)
        else:
            logger.warning("Anthropic API key not found")
            self.anthropic_client = None
        {% endif -%}

        logger.info(f"Initialized agent '{name}' with model {model}")

    def run(self, prompt: str) -> str:
        """
        Run the agent with a prompt

        Args:
            prompt: User input prompt

        Returns:
            Agent response
        """
        logger.info(f"Agent '{self.name}' received prompt")

        # Add to conversation history
        self.conversation_history.append({
            "role": "user",
            "content": prompt

```

```

    })

    # Generate response
    response = self._generate_response(prompt)

    # Add response to history
    self.conversation_history.append({
        "role": "assistant",
        "content": response
    })

    return response

def _generate_response(self, prompt: str) -> str:
    """Generate response from LLM"""

    {% if use_openai -%}
    if self.openai_client and "gpt" in self.model.lower():
        return self._generate_openai_response(prompt)
    {% endif -%}

    {% if use_anthropic -%}
    if self.anthropic_client and "claude" in self.model.lower():
        return self._generate_anthropic_response(prompt)
    {% endif -%}

    # Fallback response
    return f"Agent '{self.name}' received: {prompt}"

{% if use_openai -%}
def _generate_openai_response(self, prompt: str) -> str:
    """Generate response using OpenAI"""
    try:
        response = self.openai_client.chat.completions.create(
            model=self.model,
            messages=self.conversation_history,
            temperature=self.temperature,
            max_tokens=self.max_tokens
        )
        return response.choices[0].message.content
    except Exception as e:
        logger.error(f"OpenAI API error: {e}")
        raise
{% endif -%}

{% if use_anthropic -%}
def _generate_anthropic_response(self, prompt: str) -> str:
    """Generate response using Anthropic"""
    try:
        response = self.anthropic_client.messages.create(
            model=self.model,
            max_tokens=self.max_tokens,
            temperature=self.temperature,
            messages=self.conversation_history
        )
        return response.content[0].text
    except Exception as e:
        logger.error(f"Anthropic API error: {e}")
        raise
{% endif -%}

```

```

        except Exception as e:
            logger.error(f"Anthropic API error: {e}")
            raise
    {% endif -%}

def reset(self):
    """Reset conversation history"""
    self.conversation_history = []
    logger.info(f"Agent '{self.name}' conversation history reset")

```

File: zul/templates/agent/pyproject.toml.j2

```

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "{{ package_name }}"
version = "{{ version }}"
description = "{{ project_name_title }}" - AI Agent Application"
requires-python = ">=3.9"
dependencies = [
    {% if use_openai -%}
    "openai>=1.0.0",
    {% endif -%}
    {% if use_anthropic -%}
    "anthropic>=0.18.0",
    {% endif -%}
    "pydantic>=2.0.0",
    "pydantic-settings>=2.0.0",
    "python-dotenv>=1.0.0",
]

[project.optional-dependencies]
dev = [
    {% if use_tests -%}
    "pytest>=7.0.0",
    "pytest-asyncio>=0.21.0",
    {% endif -%}
    "black>=23.0.0",
    "ruff>=0.1.0",
    "mypy>=1.0.0",
]

[project.scripts]
{{ package_name }} = "{{ package_name }}.main:main"

[tool.black]
line-length = 100
target-version = ['py39', 'py310', 'py311', 'py312']

[tool.ruff]
line-length = 100
select = ["E", "F", "I", "N", "W"]

```

```
[tool.mypy]
python_version = "3.9"
warn_return_any = true
warn_unused_configs = true
disallow_untyped_defs = false
```

File: zul/templates/agent/README.md.j2

```
# {{ project_name_title }}

AI Agent application generated with Zul

### Features

- {{ 'OpenAI GPT' if use_openai else '' }}{{ ' and ' if use_openai and use_anthropic else '' }}
- ⚙️ Type-safe configuration with Pydantic
- 🗨️ Conversation history management
- {{ 'Testing setup with pytest' if use_tests else '' }}
- {{ 'Docker support' if use_docker else '' }}

### Installation
```

Create virtual environment

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
```

Install dependencies

```
pip install -e .
```

Install dev dependencies

```
pip install -e ".[dev]"
```

```
### Configuration

Copy `.env.example` to `.env`:
```

```
cp .env.example .env
```

```
Edit `.env` and add your API keys:
```

```
{% if use_openai -%}  
OPENAI_API_KEY=your_openai_key_here  
{% endif -%}  
{% if use_anthropic -%}  
ANTHROPIC_API_KEY=your_anthropic_key_here  
{% endif -%}  
MODEL_NAME=gpt-4  
TEMPERATURE=0.7  
MAX_TOKENS=1000
```

```
### Usage
```

```
Run the agent:
```

```
{{ package_name }}
```

```
Or as Python module:
```

```
python -m {{ package_name }}.main
```

```
### Development
```

```
{% if use_tests -%}  
Run tests:
```

```
pytest
```

```
{% endif -%}
```

```
Format code:
```

```
black .  
ruff check . --fix
```

```
Type checking:
```

```
mypy {{ package_name }}
```

```
## Project Structure
```

```
{{ package_name }}/
├── {{ package_name }}/
│   ├── init.py
│   ├── main.py # Entry point
│   ├── config.py # Configuration
│   ├── agents/
│   │   ├── init.py
│   │   └── base_agent.py # Agent implementation
│   ├── tools/ # Agent tools
│   │   ├── init.py
│   │   └── base_tool.py
│   └── utils/ # Utilities
│       └── init.py
├── tests/ # Tests
├── configs/ # Configuration files
├── data/ # Data files
├── .env.example
├── pyproject.toml
└── README.md
```

```
## License
```

```
MIT
```

```
---
```

```
Generated with [Zul](https://github.com/yourusername/zul) 〇
```

File: zul/templates/agent/.env.example.j2

```
# {{ project_name_title }} Configuration
```

```
{% if use_openai -%}
```

```
# OpenAI API
```

```
OPENAI_API_KEY=sk-...
```

```
OPENAI_ORG_ID=org-...
```

```
{% endif -%}
```

```
{% if use_anthropic -%}
```

```
# Anthropic API
```

```
ANTHROPIC_API_KEY=sk-ant-...
```

```
{% endif -%}
```

```
# Model Configuration
```

```
MODEL_NAME=gpt-4
TEMPERATURE=0.7
MAX_TOKENS=1000

# Application
LOG_LEVEL=INFO
DEBUG=False
ENVIRONMENT=development
```

File: zul/templates/agent/.gitignore.j2

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg

# Virtual environments
venv/
ENV/
env/
.venv

# IDE
.vscode/
.idea/
*.swp
*.swo
*~

# Environment
.env
.env.local

# Testing
.pytest_cache/
.coverage
htmlcov/
```

```
# Data
data/*.csv
data/*.json
!data/.gitkeep

# Logs
*.log
logs/
```

Langkah 6: Buat Init Command

File: zul/commands/init.py

```
"""
Init command - Initialize template configuration
"""
import typer
from rich.console import Console
from rich import print as rprint
from InquirerPy import inquirer

app = typer.Typer(help="Initialize Zul configuration")
console = Console()

@app.command()
def config():
    """Initialize Zul configuration file"""

    console.print("[cyan]Initializing Zul configuration...[/cyan]\n")

    # Interactive setup
    default_author = inquirer.text(
        message="Your name (for generated projects):",
        default="Developer"
    ).execute()

    default_email = inquirer.text(
        message="Your email:",
        default="[email protected]"
    ).execute()

    default_output = inquirer.text(
        message="Default output directory:",
        default="."
    ).execute()

    # Save configuration
    config_content = f"""# Zul Configuration
AUTHOR_NAME={default_author}
AUTHOR_EMAIL={default_email}
DEFAULT_OUTPUT_DIR={default_output}
"""

    from pathlib import Path
```

```
config_file = Path.home() / ".zulrc"
config_file.write_text(config_content)

console.print(f"\n[green]✓[/green] Configuration saved to {config_file}")
```

Langkah 7: Testing Tool Anda

Sekarang test tool zul yang sudah dibuat:

```
# Install dalam editable mode
pip install -e .

# Test help
zul --help

# Test build command
zul build --help

# Generate AI agent project (interactive)
zul build agent

# Generate dengan options
zul build agent --name my_awesome_agent --output ~/projects

# Test generated project
cd my_awesome_agent
python -m venv venv
source venv/bin/activate
pip install -e .

# Setup environment
cp .env.example .env
# Edit .env dan tambahkan API keys

# Run agent
python -m my_awesome_agent.main
```

Ringkasan Fitur yang Telah Dibuat

✓ **CLI Framework dengan Typer:** [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)

- Subcommands structure (build, init)
- Rich output dengan colors dan panels
- Auto-generated help text
- Type hints untuk IDE support

✓ **Interactive Prompts dengan InquirerPy:** [\[5\]](#) [\[6\]](#)

- Text input dengan validation
- Confirm prompts
- Select menus

- Path validation

✓ **Beautiful Output dengan Rich:** [\[7\]](#) [\[8\]](#) [\[9\]](#) [\[10\]](#)

- Colored console output
- Progress spinners
- Panels dan formatting
- Tables (bisa ditambahkan)

✓ **Template Engine dengan Jinja2:** [\[11\]](#) [\[12\]](#) [\[13\]](#)

- Variable substitution
- Conditional rendering (`{% if %}`)
- Template inheritance
- Custom filters

✓ **Type-Safe Config dengan Pydantic Settings:** [\[14\]](#) [\[15\]](#)

- Environment variable loading
- Type validation
- Default values
- .env file support

Next Steps untuk Enhancement

1. **Add More Templates:** API, CLI, Data Science, dll
2. **Template Updates:** Implement update command untuk existing projects
3. **Custom Hooks:** Pre/post generation scripts
4. **Plugin System:** Allow community templates
5. **Testing:** Add comprehensive test suite
6. **CI/CD:** GitHub Actions untuk automated testing
7. **Documentation:** Detailed docs dengan MkDocs

Tool zu1 Anda sekarang sudah siap digunakan! ☐



1. <https://calmcode.io/course/typer/subcommands>
2. <https://typer.tiangolo.com/tutorial/subcommands/>
3. <https://typer.tiangolo.com/tutorial/subcommands/nested-subcommands/>
4. <https://typer.tiangolo.com/tutorial/subcommands/single-file/>
5. <https://inquirerpy.readthedocs.io>
6. <https://inquirerpy.readthedocs.io/en/latest/>
7. <https://rich.readthedocs.io/en/latest/tables.html>

8. <https://github.com/Textualize/rich>
9. <https://realpython.com/python-rich-package/>
10. <https://rich.readthedocs.io/en/latest/console.html>
11. <https://copier.readthedocs.io/en/stable/creating/>
12. <https://copier.readthedocs.io/en/stable/configuring/>
13. <https://jinja.palletsprojects.com/en/stable/templates/>
14. https://docs.pydantic.dev/latest/concepts/pydantic_settings/
15. <https://docs.pydantic.dev/1.10/usage/settings/>
16. <https://stackoverflow.com/questions/54718238/jinja2-copy-variable-instead-of-referencing-it>
17. <https://github.com/copier-org/copier-templates-extensions>
18. <https://typer.tiangolo.com/tutorial/subcommands/add-typer/>
19. <https://www.freecodecamp.org/news/use-the-rich-library-in-python/>
20. <https://typer.tiangolo.com/tutorial/using-click/>
21. <https://github.com/copier-org/copier/issues/229>
22. <https://rich.readthedocs.io/en/stable/reference/console.html>
23. <https://typer.tiangolo.com/tutorial/subcommands/name-and-help/>
24. [https://stackoverflow.com/questions/73388551/using-jinja2-template-to-copy-content-in-a-file-with-an-
isble](https://stackoverflow.com/questions/73388551/using-jinja2-template-to-copy-content-in-a-file-with-an-
<u>isble</u>)